CMPT 295

Unit – Microprocessor Design & Instruction Execution

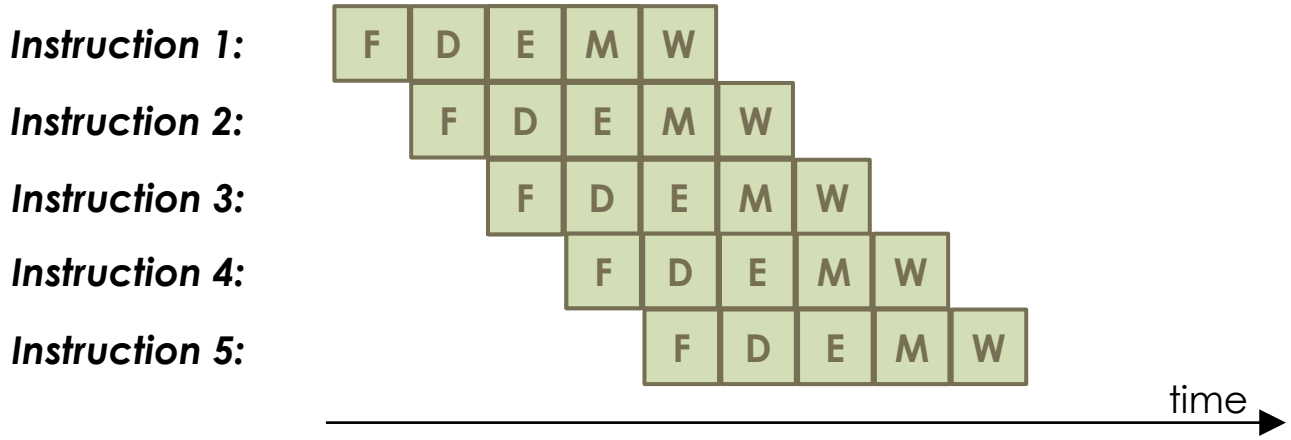
Lecture 34 – Pipelined Execution and Hazards – cont'd

Last Lecture - 1

Microprocessor machine code instruction execution:

Model 3. Pipelined execution of machine code instructions

- Start executing 1 instruction at each clock cycle
- Effect: overlap different stages as different instructions are executing



Steady state:
With **n** stages,
now executing
n instructions in
n clock cycles

- Analysis of pipelined execution using best case pipeline scenario, i.e., all stages have the same propagation delay, and "steady-state"
  - **Result**: Higher throughput

Last Lecture - 2

Microprocessor machine code instruction execution:

3. Pipelined execution of machine code instructions

- Limitations on pipelined execution
  1. Doubling the number of stages does not double the throughput
  2. Non-uniform propagation delay of stages: all stages may not have the same propagation delay – stage with longest propagation delay (slowest stage) limits clock cycle
  3. Hazard: 1) Structural, 2) Data and 3) Control
     1. Structural Hazard: when 2 stages attempt to use same hardware resource at same time, e.g., memory
        - Solution: MIPS has *Data Memory* and *Instruction Memory*
     2. Data Hazard created by data dependencies
        - Solution 1: Stall (bubbles)
        - Solution 2: Data forwarding
        - Specific type of Data Hazard: **Load-to-use**
          - Solution 1: Load interlock
          - Solution 2: Interleaving (reordering) instructions

Today's Menu

- Instruction Set Architecture (ISA)
  - Definition of ISA
- Instruction Set design
  - Design principles
  - Look at an example of an instruction set: MIPS
  - Create our own
  - ISA evaluation
- Implementation of a microprocessor (CPU) based on an ISA
  - Execution of machine code instructions (datapath)
  - Intro to logic design + Combinational logic + Sequential logic circuit
  - Sequential execution of machine code instructions
  - Staged execution of machine code instructions
  - Pipelined execution of machine code instructions + Hazards (cont'd)

Sequential Instructions

- When executing sequential instructions, the address of next instruction (next value of PC) is predictable: PC + length of current instruction
  - For example:

```
lw  $s2, 4($sp)
add $s3, $s1, $s2
sub $s4, $s1, $s2
mul $s5, $s3, $s4
sw  $s5, 8($sp)
```

- Therefore, in those situations, the goal of executing 1 instruction per clock cycle is achieved!

3rd Class of Instructions

- But we do not always execute sequential instructions
- Sometime, we execute a function call or a **`goto`** ☹:
  - **`call`** and **`jump`** – both *unconditional*
    - For example: `call` **`proc`**, where **`proc`** is a label (a.k.a. the *branch target*) and a label is a memory address
      - In this example, the memory address of the next instruction (next value of PC), i.e., the label **`proc`**, is set in stage **D**, when the value of **`proc`** (memory address) is extracted from one of the fields of the machine code instruction related to a **`call`** assembly code instruction
  - **`ret`**
    - Here, the memory address of the next instruction (next value of PC), which is the memory address of the instruction that comes after the function call, is set in stage **M**, since this memory address is popped from stack

3rd Class of Instructions (cont'd)

- But we do not always execute sequential instructions
- Sometime, we execute a conditional or an iterative statement where a **condition** dictates which statements are executed:
  - **`jump`** - conditional
    - For example: `jle loop`, where **`loop`** is a label (a.k.a. the *branch target*) and a label is a memory address
      1. In this example, the memory address of the next instruction (next value of PC) is either
         - set in stage **D**, when the value of **`loop`** (memory address) is extracted from one of the fields of the machine code instruction related to a conditional **`jump`** assembly code instruction (here: `jle`) - if **`jump`** is executed (conditions `le`)

         OR

         - set to PC + length of instruction - if **`jump`** is not executed (condition `g`)
      2. And, the decision to branch, i.e., to **`jump`** or not, is done in stage **E** when the condition codes are ascertained (are the conditions set to `le` or `g`?)

Hazard 3 - Control Hazard

- All of these instructions: **`call`** and *unconditional* **`jump`**, **`ret`**, *conditional* **`jump`**, are referred to as branch instructions
- Let's now focus on the *conditional* **`jump`** branch instructions
- When executing branch instructions, like any other instructions, the "correct next" instruction must be fetched at the next clock cycle
  - Remember, our goal in designing this pipelined microprocessor is to execute 1 instruction per clock cycle, i.e., fetch a new instruction at every "tick" of the clock!
- But this "correct next" instruction is determined based on whether microprocessor is branching or not, which, as we saw on previous slide, is determined in later stages (later than stage **F**)
  - So, if the microprocessor is currently fetching a branch instruction in stage **F**, it will not yet know whether it will branch or not, i.e., it will not yet know which instruction to fetch at the next clock cycle
- Possible solutions:
  1. Stall the pipeline until microprocessor knows
     - Issue: slows down microprocessor
  2. Microprocessor ***predicts*** (guesses) whether it will be branching or not

Branch Prediction Strategy

- Definition: Method of resolving control hazard
  - Microprocessor predicts a particular outcome: jump or not (branch taken or branch not taken) for the branch instruction that is being fetched/decoded and proceeds on setting the PC based on that ***prediction*** rather than wait for branch instruction to have started its execution
- Used by virtually all microprocessors
- Lots of research/experiments on effective branch prediction strategies (dynamic branch predictors)
- Usually 90%+ correct
- Examples of *simple branch prediction strategies*:
  1. Conditional branches are **always taken**, i.e., always jump
     - So new PC value set to memory address obtained from instruction
  2. Conditional branches are **never taken**, i.e., never jump
     - So new PC value -> PC + length of instruction

What if the microprocessor *mis-predicts*?

Let's have a look at an example:

```
for (i=0; i<10; i++)
    sum += i;
```

- Branch prediction strategy 1: "always jump" is correct ->
  is wrong ->
- Branch prediction strategy 2: "never jump" is correct ->
  is wrong ->
- When the prediction is WRONG:
  - Instructions being fetched/decoded must be discarded -> clock cycles wasted ☹
- When this prediction is CORRECT:
  - All is well! -> achieve 1 instruction per clock cycle ☺

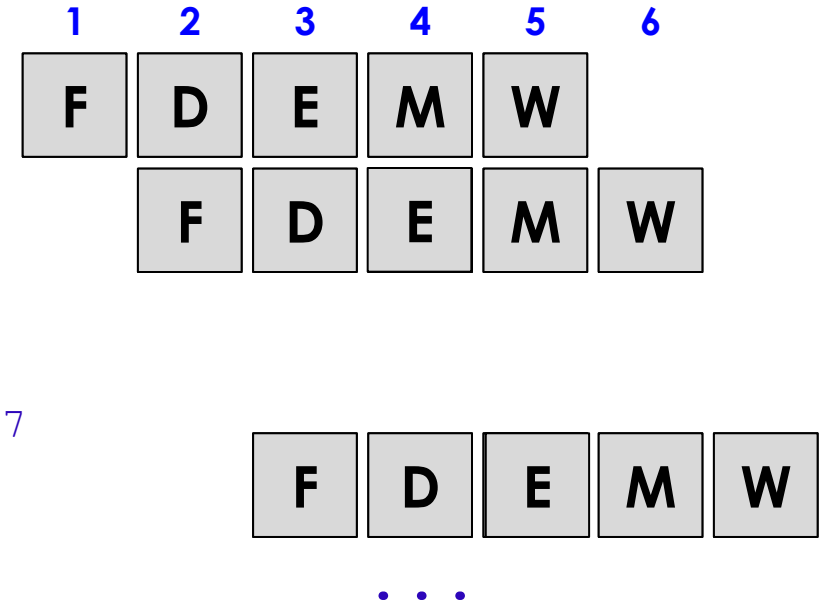
What happens when branch prediction strategy is *conditional branches are **always taken**, i.e., always jump*

Branching - 1

- When the next instruction in the pipeline is a branch, which instruction will be fetched next?
  - Either PC+length of instruction -> I_3 or branch target -> I_8, depending on condition codes
  - Condition: **`i < 10`**

```
i  -> r1
10 -> r2
```

```
I1: CMP r2, r1
I2: Jl loop
I3: SUB r7, r6
I4: ... + I5,I6,I7
I8: loop: ADD r1, r3
I9: ...
```

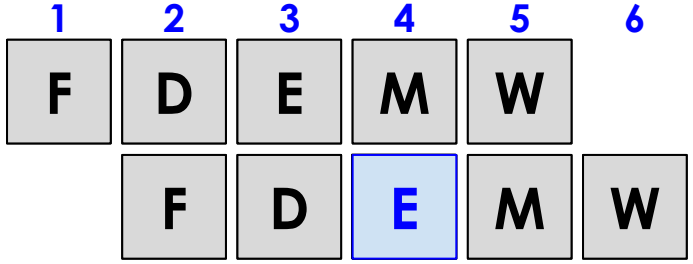


What happens when prediction is correct
i.e., when `i has value [0..9]`

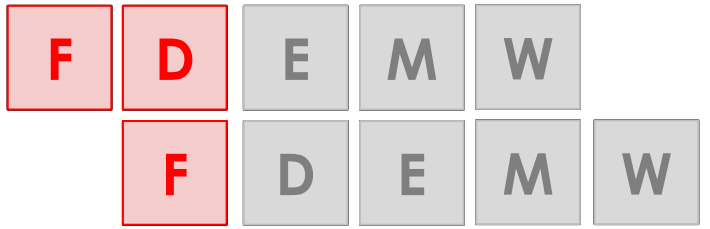
Branch prediction

- Result of branch not known until stage **E** of `J1` at clock cycle 4
- But the decision (*which instruction to execute next*) needs to be made in time for clock cycle 3
- So processor makes a decision: it predicts that `i < 10`
  -> so it is going to jump (***branch is always taken***)

```
I1:  CMP r2, r1
I2:  J1 loop
I3:  SUB r7, r6
I4:  ... + I5,I6,I7
I8:  loop: ADD r1, r3
I9:  ...
I10: ...
```



Speculative execution



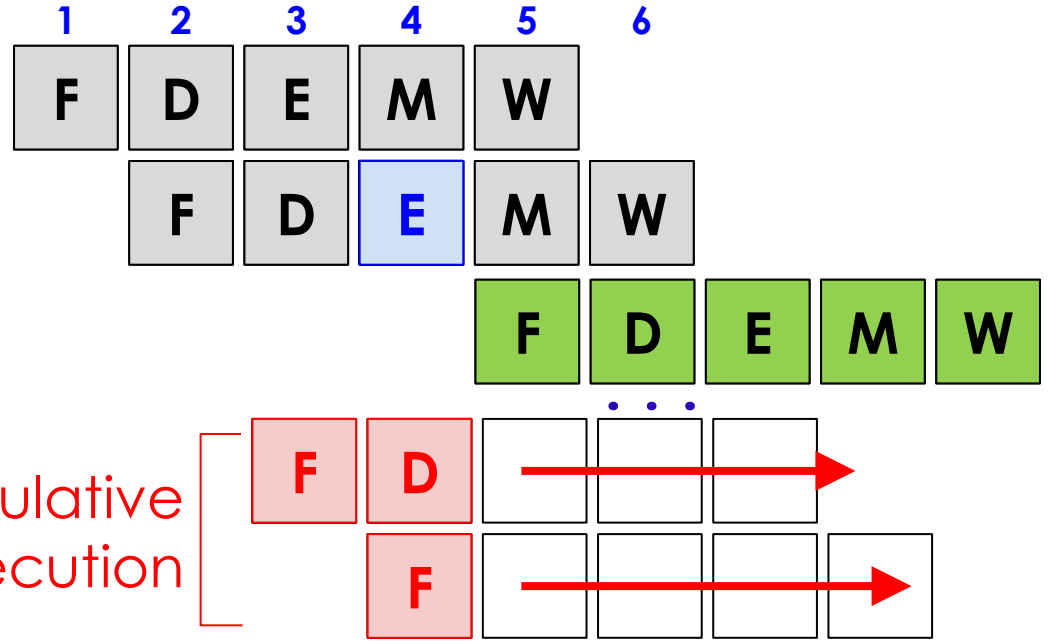
Execution no longer speculative

What happens when prediction is wrong
i.e., when **`i has value 10`**

Branch mis-predictions

- What happens when microprocessor made the wrong decision (it mis-predicted)
- In stage **E** of **`J1`** at clock cycle 4, because **`i == 10 ->`** ~~`J1`**
  -> should not have jumped (branch should not have been taken)
  - Then I_8 and I_9 are squashed, i.e, discarded

```
I1: CMP r2, r1
I2: J1 loop
I3: SUB r7, r6
I4: ... + I5,I6,I7
I8: loop: ADD r1, r3
I9: ...
```



Conclusion: Mis-predicting branches produces loss of clock cycles (↓throughput)

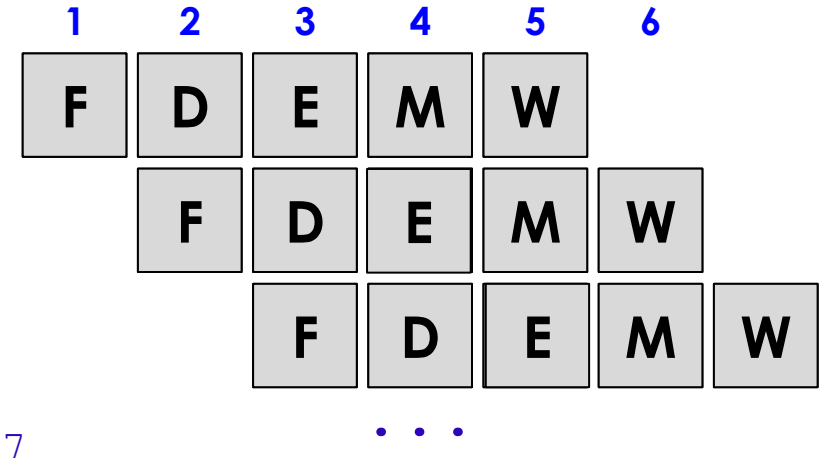
What happens when branch prediction strategy is *conditional branches are **never taken**, i.e., never jump*

Branching - 2

- When the next instruction in the pipeline is a branch, which instruction will be fetched next?
  - Either PC+length of instruction -> I_3 or branch target -> I_8, depending on condition codes
  - Condition: **`i < 10`**

```
i  -> r1
10 -> r2
```

```
I1: CMP r2, r1
I2: Jl loop
I3: SUB r7, r6
I4: ... + I5,I6,I7
I8: loop: ADD r1, r3
I9: ...
```



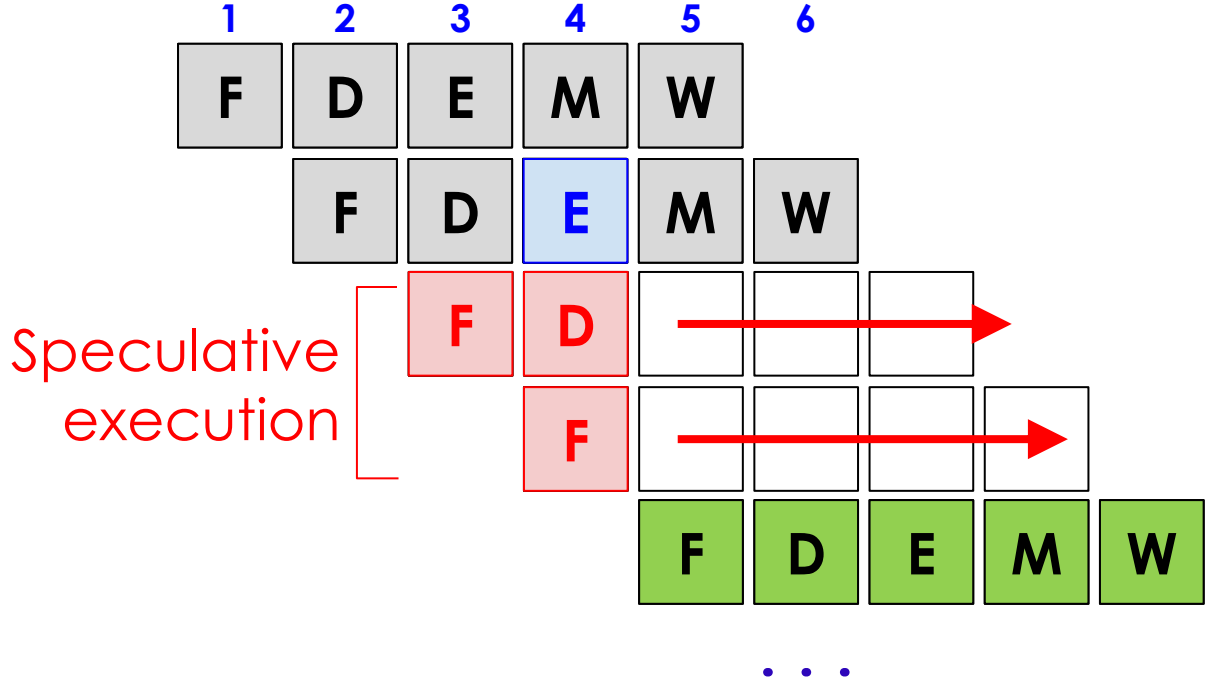
What happens when prediction is wrong
i.e., when **`i has value [0..9]`**

Branch mis-predictions

- What happens when microprocessor makes the wrong decision (it mis-predicts)
- In stage **E** of **J1** at clock cycle 4, because **`i < 10 ->`** J1 -> should have jumped (branch should have been taken)
  - Then I_3 and I_4 are squashed, i.e, discarded

```
I1: CMP r2, r1
I2: J1 loop
I3: SUB r7, r6
I4: ... + I5,I6,I7

I8: loop: ADD r1, r3
I9: ...
```



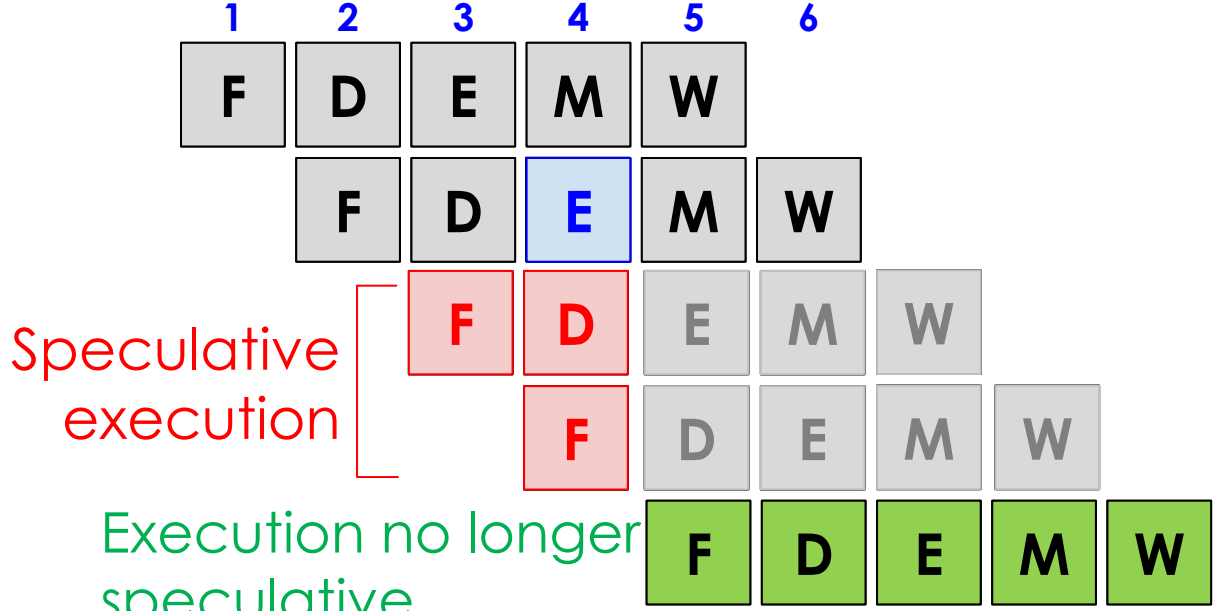
Conclusion: Mis-predicting branches produces loss of clock cycles (↓throughput)

What happens when prediction is correct
i.e., when `i has value 10`

Branch prediction

- Result of branch not known until stage **E** of `J1` at clock cycle 4
- But the decision (*which instruction to execute next*) needs to be made in time for clock cycle 3
- So processor makes a decision: it predicts that `i == 10`
  -> so it is not going to jump (***branch never taken***)

```
I1: CMP r2, r1
I2: J1 loop
I3: SUB r7, r6
I4: ...
I5: ... + I6, I7
I8: loop: ADD r1, r3
I9: ...
```



Summary

Microprocessor machine code instruction execution:

3. Pipelined execution of machine code instructions

- Limitations on pipelined execution

  …

  3. Hazard: 1) Structural, 2) Data and 3) Control

     3. Control hazard created when microprocessor needs to guess whether to branch/jump or not. The hazard is that the microprocessor may get it wrong, i.e., it may mis-predict. This causes instructions to be squashed, negatively affecting the throughput

        - Solution:

          - Simple branch prediction strategies: **always jump** or **never jump**

Next Lecture

- Code optimization
  - Examples of code optimization
    - Done by compilers
    - Done by software developers
  - More optimization strategies - Based on microprocessor architecture
    - Loop unrolling
    - Instruction parallelism
      - Re-association
      - Accumulators